November 6, 2026 at 22:52

1. Introduction. Exercise 7.2.2.2–212 is the last link of a chain of reductions that begins in exercise 204 and shows that three innocent-looking problems about latin squares are NP-complete. The two exercises on page 335 read:

> **211.** **[30]** (R. W. Irving and M. Jerrum, 1994.) Use exercise 208 to reduce 3SAT to the problem of list coloring a grid graph of the form $K_N \,\square\, K_3$. (Hence the latter problem, which is also called *latin rectangle construction*, is NP-complete.)
>
> **212.** **[32]** Continuing the previous exercise, we shall reduce grid list coloring to another interesting problem called *partial latin square construction*. Given three $n \times n$ binary matrices (r_{ik}), (c_{jk}), (p_{ij}), the task is to construct an $n \times n$ array (X_{ij}) such that X_{ij} is blank when $p_{ij} = 0$, otherwise $X_{ij} = k$ for some k with $r_{ik} = c_{jk} = 1$; furthermore the nonblank entries must be distinct in each row and column.
> a) Show that this problem is symmetrical in all three coordinates: It's equivalent to constructing a binary $n \times n \times n$ tensor (x_{ijk}) such that $x_{*jk} = c_{jk}$, $x_{i*k} = r_{ik}$, and $x_{ij*} = p_{ij}$, for $1 \le i, j, k \le n$, where '$*$' denotes summing an index from 1 to n. (Therefore it is also known as the *binary $n \times n \times n$ contingency problem*, given n^2 row sums, n^2 column sums, and n^2 pile sums.)
> b) A necessary condition for solution is that $c_{*k} = r_{*k}$, $c_{j*} = p_{*j}$, and $r_{i*} = p_{i*}$. Exhibit a small example where this condition is not sufficient.
> c) If $M < N$, reduce $K_M \,\square\, K_N$ list coloring to the problem of $K_N \,\square\, K_N$ list coloring.
> d) Finally, explain how to reduce $K_N \,\square\, K_N$ list coloring to the problem of constructing an $n \times n$ partial latin square, where $n = N + \sum_{I,J} |L(I, J)|$. *Hint:* Instead of considering integers $1 \le i, j, k \le n$, let i, j, k range over a *set* of n elements. Define $p_{ij} = 0$ for most values of i and j; also make $r_{ik} = c_{ik}$ for all i and k.

2. The answers are on page 589, and answer 211 leans on answers 204(a), 207, and 208 on pages 587–588. This program checks all of them, in the order in which the chain uses them.

1. *The ladder.* Answer 204(a) makes every variable consistent around a cycle of binary clauses; answer 207 builds twenty unsatisfiable clauses in which each of 30 literals occurs twice; answer 208 combines the two into an equivalent problem where every literal occurs exactly twice.
2. *The grid.* Answer 211 turns such a problem, with $3m = 4n$, into lists for $K_N \,\square\, K_3$ with $N = 8n$, colorable exactly when the clauses are satisfiable.
3. *The tensor.* Answer 212(a) identifies x_{ijk} with $[X_{ij} = k]$.
4. *The small example.* Answer 212(b) gives a 4×4 instance.
5. *The extension.* Answer 212(c) fills the missing rows with full lists, by Theorem 7.5.1L.
6. *The square.* Answer 212(d) builds the partial latin square with headers.
7. *The whole chain.* Finally the reductions are composed, from 3SAT to a partial latin square with more than twelve thousand rows, and the `sat` package is asked to solve the result.

Wherever the objects are small, the program counts solutions by brute force, with the dancing-cells exact cover solver; where they are not, the package answers, and its answers are decoded back to the original problem and checked there. Constructions that the answers describe in the forward direction are also carried out in that direction, without a solver.

I wrote this on 16 September 2026.

3. Every claim is a mode.

```
package main
import (
    "context"
    "flag"
    "fmt"
    "math/rand/v2"
    "os"
    "slices"
    "strconv"
    "strings"
    cells "github.com/sjnam/dancing-cells"
    zdd "github.com/sjnam/dancing-cells/zdd"
    "github.com/sjnam/sat"
)
⟨ Types 8 ⟩
⟨ Global variables 5 ⟩
⟨ Functions 6 ⟩
func main( ) {
    ⟨ Read the command line 4 ⟩
    if on("ladder") {
        ⟨ Check answers 204(a), 207, and 208 24 ⟩
    }
    if on("grid") {
        ⟨ Check answer 211 42 ⟩
    }
    if on("tensor") {
        ⟨ Check answer 212(a) 56 ⟩
    }
    if on("small") {
        ⟨ Check answer 212(b) 58 ⟩
    }
    if on("extend") {
        ⟨ Check answer 212(c) 67 ⟩
    }
    if on("square") {
        ⟨ Check answer 212(d) 73 ⟩
    }
    if on("whole") {
        ⟨ Run the whole chain through the sat package 77 ⟩
    }
    ⟨ Say how it went 7 ⟩
}
```

4. ⟨ Read the command line 4 ⟩ ≡

```
flag.StringVar(&mode, "mode", "all",
    "ladder, grid, tensor, small, extend, square, whole, or all")
flag.IntVar(&reps, "reps", 200, "how many random cases per test")
flag.Uint64Var(&seed, "seed", 20260916, "seed for the random cases")
flag.Parse()
```

This code is used in section 3.

5. ⟨ Global variables 5 ⟩ ≡

```
var (
    mode string
    reps int
    seed uint64
    failures int
)
```

This code is used in section 3.

6. Each claim is announced, checked, and marked; one `FAIL` anywhere makes the program exit with a nonzero status.

⟨ Functions 6 ⟩ ≡

```
func on(m string) bool { return mode ≡ m ∨ mode ≡ "all" }
func claim(ok bool, format string, args ... any) {
    mark := "ok  "
    if ¬ok {
        mark = "FAIL"
        failures++
    }
    fmt.Printf("  %s  %s\n", mark, fmt.Sprintf(format, args...))
}
func rng(stream uint64) *rand.Rand { return rand.New(rand.NewPCG(seed, stream)) }
```

This code is also defined in sections 9, 10, 11, 13, 14, 15, 17, 18, 19, 20, 21, 22, 31, 33, 34, 35, 37, 38, 39, 41, 52, 54, 55, 62, 63, 64, 66, 69, 71, 72.

This code is used in section 3.

7. ⟨ Say how it went 7 ⟩ ≡

```
if failures > 0 {
    fmt.Printf("\n%d claims failed.\n", failures)
    os.Exit(1)
}
fmt.Println("\nEvery claim checked out.")
```

This code is used in section 3.

8. Clauses. Literals are numbered as Knuth numbers them, $2v$ for v and $2v+1$ for $\bar{v}$, with variables $1 \le v \le n$. An assignment is a bit vector with bit v for variable v. The problems of this program that are decided by brute force have at most fifteen variables.

⟨ Types 8 ⟩ ≡

```
type cnf struct {
    n int
    cl [][]int
}
```

This code is also defined in sections 12, 30, 51.

This code is used in section 3.

9. Clauses are written as in the book, one after another, separated by commas; the literals of a clause are separated by spaces, since some variables have two digits, and a prime marks a complemented literal.

⟨ Functions 6 ⟩ +≡

```
func parseCNF(n int, s string) cnf {
    f := cnf{n: n}
    for _, c := range strings.Split(s, ",") {
        var cl []int
        for _, w := range strings.Fields(c) {
            v, _ := strconv.Atoi(strings.TrimSuffix(w, "'"))
            cl = append(cl, 2 * v + strings.Count(w, "'"))
        }
        f.cl = append(f.cl, cl)
    }
    return f
}
func (f cnf) String() string {
    var cs []string
    for _, c := range f.cl {
        var ls []string
        for _, l := range c {
            ls = append(ls, strconv.Itoa(l >> 1) + strings.Repeat("'", l & 1))
        }
        cs = append(cs, strings.Join(ls, "␣"))
    }
    return strings.Join(cs, ",␣")
}
func holds(l int, a uint64) bool { return (a >> (l >> 1) & 1 == 1) == (l & 1 == 0) }
func satisfies(f cnf, a uint64) bool {
    for _, c := range f.cl {
        if ¬slices.ContainsFunc(c, func(l int) bool { return holds(l, a) }) {
            return false
        }
    }
    return true
}
func models(f cnf) []uint64 {
    var ms []uint64
    for x := uint64(0); x < 1 << f.n; x++ {
        if satisfies(f, x << 1) {
            ms = append(ms, x << 1)
        }
    }
    return ms
}
```

10. The literal counts, which several answers are about.

⟨ Functions 6 ⟩ +≡

```
func occurrences(f cnf) []int {
    occ := make([]int, 2 * f.n + 2)
    for _, c := range f.cl {
        for _, l := range c {
            occ[l]++
        }
    }
    return occ
}
```

11. The package decides the larger problems.

⟨ Functions 6 ⟩ +≡

```
func solveCNF(f cnf, assumptions ...int) bool {
    s := sat.New()
    for v := 1; v ≤ f.n; v++ {
        s.NewVar()
    }
    for _, c := range f.cl {
        ls := make([]sat.Lit, len(c))
        for i, l := range c {
            ls[i] = sat.Lit(l)
        }
        s.AddClause(ls...)
    }
    as := make([]sat.Lit, len(assumptions))
    for i, l := range assumptions {
        as[i] = sat.Lit(l)
    }
    st, _ := s.Solve(context.Background(), as...)
    return st ≡ sat.Sat
}
```

12. Exact cover. Small instances of every problem in this chain are exact cover problems: a list coloring chooses one color per cell, and a partial latin square one triple (i, j, k) per nonblank cell, with each row–color and column–color pair used at most once, or exactly once. They are solved with `github.com/sjnam/dancing-cells`, Knuth's dancing cells, the package behind the companion readings of Section 7.2.2.1; so these checks test that package too. Two of its engines serve. The XCC engine finds covers one at a time, which is what a search for one cover, or a look at every cover, wants. The ZDD engine gathers all covers into a decision diagram, remembering every subproblem it has solved, and counts them from the diagram; on the grids of answer 211, with hundreds of thousands of colorings, it counts about a hundred times faster than ranging over the covers would.

An **xcover** collects items numbered from 0, the first *primary* of them primary, and options given as lists of item numbers.

⟨ Types 8 ⟩ +≡

```
type xcover struct {
    primary, items int
    opts [][]int
}
```

13. ⟨ Functions 6 ⟩ +≡

```
func newCover(primary, items int) *xcover {
    return &xcover{primary: primary, items: items}
}
func (x *xcover) add(items ... int) { x.opts = append(x.opts, items) }
```

14. Both engines read the DLX format. *dlx* writes the problem in it, item t named `it`, and indexes the options by their text, because the XCC engine reports an option by its item names in input order. A line that begins with a vertical bar is a comment in that format, so a problem with no primary items cannot be written down; but every option here has a primary item, so such a problem has no options, and its only solution is the empty one. The two callers below deal with that case before asking for the text.

⟨ Functions 6 ⟩ +≡

```
func (x *xcover) dlx()(string, map[string]int) {
    ⟨ Write x in DLX format, and index its options by their text 16 ⟩
    return b.String(), index
}
```

15. *search* calls *visit* with the options of every solution, as indices, until *visit* returns `false`; cancelling the context then stops the solver.

⟨ Functions 6 ⟩ +≡

```
func (x *xcover) search(visit func(sol []int) bool) {
    if x.primary ≡ 0 {
        visit(Λ)
        return
    }
    text, index := x.dlx()
    ctx, cancel := context.WithCancel(context.Background())
    defer cancel()
    res := cells.NewXCC().WithContext(ctx).Dance(strings.NewReader(text))
    for opts := range res.Solutions {
        sol := make([]int, len(opts))
        for i, o := range opts {
            sol[i] = index[strings.Join(o, "␣")]
        }
        if ¬visit(sol) {
            return
        }
    }
}
```

16. ⟨ Write x in DLX format, and index its options by their text 16 ⟩ ≡

```
var b strings.Builder
for t := 0; t < x.items; t++ {
    if t ≡ x.primary {
        b.WriteString("␣|")
    }
    fmt.Fprintf(&b, "␣i%d", t)
}
b.WriteByte('\n')
index := make(map[string]int, len(x.opts))
for o, items := range x.opts {
    names := make([]string, len(items))
    for i, it := range items {
        names[i] = "i" + strconv.Itoa(it)
    }
    line := strings.Join(names, "␣")
    index[line] = o
    b.WriteString(line + "\n")
}
```

This code is used in section 14.

17. Whether a cover exists is a question for the XCC engine, which stops at the first one; how many there are is a question for the ZDD engine.

⟨ Functions 6 ⟩ +≡

```
func (x *xcover) exists() bool {
    found := false
    x.search(func([]int) bool {
        found = true
        return false
    })
    return found
}
func (x *xcover) total() int {
    if x.primary ≡ 0 {
        return 1
    }
    text, _ := x.dlx()
    return int(zdd.New().Dance(strings.NewReader(text)).Count().Int64())
}
```

18. The ladder: answers 204(a), 207, and 208. Answer 204(a) is due to C. A. Tovey. The m clauses become $(X_1 \vee X_2 \vee X_3)$, ..., and if the literals involving x_j are $\sigma_1 X_{i(1)}$, ..., $\sigma_p X_{i(p)}$, the clauses $(-\sigma_h X_{i(h)} \vee \sigma_{h^+} X_{i(h^+)})$ with $h^+ = 1 + (h \bmod p)$ make them agree. Here X_i is variable i, the occurrence in position i counted from 1, and the binary clauses come out variable by variable, in the order of appearance.

⟨ Functions 6 ⟩ +≡

```
func tovey(f cnf) cnf {
    m := len(f.cl)
    g := cnf{n: 3 * m}
    occ := make([][]int, f.n + 1)
    for i := 0; i < 3 * m; i++ {
        occ[f.cl[i/3][i % 3] >> 1] = append(occ[f.cl[i/3][i % 3] >> 1], i)
    }
    for k := 0; k < m; k++ {
        g.cl = append(g.cl, []int{2 * (3 * k + 1), 2 * (3 * k + 2), 2 * (3 * k + 3)})
    }
    for v := 1; v <= f.n; v++ {
        for h, i := range occ[v] {
            i2 := occ[v][(h + 1) % len(occ[v])]
            sg, sg2 := f.cl[i/3][i % 3] & 1, f.cl[i2/3][i2 % 3] & 1
            g.cl = append(g.cl, []int{2 * (i + 1) + 1 - sg, 2 * (i2 + 1) + sg2})
        }
    }
    return g
}
```

19. Answer 207 introduces the five clauses $C(x, y, z; a, b, c) = \{x\bar{a}b, y\bar{b}c, z\bar{c}a, abc, \bar{a}\bar{b}\bar{c}\}$, and the note by Iwama and Takaki uses them too.

⟨ Functions 6 ⟩ +≡

```
func gadget(x, y, z, a, b, c int) [][]int {
    return [][]int{{x, a ⊕ 1, b}, {y, b ⊕ 1, c}, {z, c ⊕ 1, a}, {a, b, c},
        {a ⊕ 1, b ⊕ 1, c ⊕ 1}}
}
```

20. The solution is $C(x, y, y; 1, 2, 3) \cup C(x, \bar{y}, \bar{y}; 4, 5, 6) \cup C(\bar{x}, z, z; 7, 8, 9) \cup C(\bar{x}, \bar{z}, \bar{z}; a, b, c)$. Its fifteen variables are numbered $x = 1$, $y = 2$, $z = 3$, then $1, \ldots, 9$ as 4 to 12 and a, b, c as 13 to 15.

⟨ Functions 6 ⟩ +≡

```
func answer207() cnf {
    f := cnf{n: 15}
    f.cl = append(f.cl, gadget(2, 4, 4, 8, 10, 12)...)
    f.cl = append(f.cl, gadget(2, 5, 5, 14, 16, 18)...)
    f.cl = append(f.cl, gadget(3, 6, 6, 20, 22, 24)...)
    f.cl = append(f.cl, gadget(3, 7, 7, 26, 28, 30)...)
    return f
}
```

21. The sixteen clauses $\{\bar{x}\bar{y}\bar{z}\} \cup C(x,x,x;1,2,3) \cup C(y,y,y;4,5,6) \cup C(z,z,z;7,8,9)$, on twelve variables numbered the same way.

⟨ Functions 6 ⟩ +≡

```
func iwamaTakaki() cnf {
    f := cnf{n: 12, cl: [][]int{{3, 5, 7}}}
    f.cl = append(f.cl, gadget(2, 2, 2, 8, 10, 12)...)
    f.cl = append(f.cl, gadget(4, 4, 4, 14, 16, 18)...)
    f.cl = append(f.cl, gadget(6, 6, 6, 20, 22, 24)...)
    return f
}
```

22. Answer 208: "Make m clones of all but one of the 20 clauses in answer 207, and put the other $3m$ cloned literals into the $3m$ binary clauses of answer 204(a). This gives $23m$ 3-clauses in which every literal occurs twice, except that the $3m$ literals $\bar{X}_i$ occur only once." Then $3m$ variables u_i pad them, with the clauses $\bar{X}_i u_i \bar{u}_{i+1}$ and $\{u'_{3j}u'_{3j+1}u'_{3j+2}, \bar{u}'_{3j}\bar{u}'_{3j+1}\bar{u}'_{3j+2}\}$, subscripts mod $3m$, where u'_i is u_i or $\bar{u}_i$ as i is even or odd.

The variables are the $3m$ X's, then $15m$ clone variables, fifteen per clone, then the u's. Clause *drop* of answer 207 is the one left out. When m is odd, reducing $3j+1$ and $3j+2$ modulo $3m$ changes their parity, and the answer does not say which parity it means; *reduced* chooses. The function also returns the $23m$ clauses before padding.

⟨ Functions 6 ⟩ +≡

```
func clones208(f cnf, drop int, reduced bool)(cnf, int) {
    m := len(f.cl)
    t := tovey(f)
    g := cnf{n: 21 * m, cl: slices.Clone(t.cl[:m])}
    var extra []int
    for r := 0; r < m; r++ {
        off := 2 * (3 * m + 15 * r)
        for i, c := range answer207().cl {
            d := []int{c[0] + off, c[1] + off, c[2] + off}
            if i ≡ drop {
                extra = append(extra, d...)
            } else {
                g.cl = append(g.cl, d)
            }
        }
    }
    for i, c := range t.cl[m:] {
        g.cl = append(g.cl, []int{c[0], c[1], extra[i]})
    }
    before := len(g.cl)
    ⟨ Pad with the variables u_i 23 ⟩
    return g, before
}
```

23. ⟨ Pad with the variables u_i 23 ⟩ ≡

```
u := func(i int) int { return 18 * m + 1 + ((i - 1) % (3 * m) + 3 * m) % (3 * m) }
up := func(i int) int {
    if reduced {
        i = 1 + ((i - 1) % (3 * m) + 3 * m) % (3 * m)
    }
    return 2 * u(i) + i & 1
}
for i := 1; i ≤ 3 * m; i++ {
    g.cl = append(g.cl, []int{2 * i + 1, 2 * u(i), 2 * u(i + 1) + 1})
}
for j := 1; j ≤ m; j++ {
    g.cl = append(g.cl, []int{up(3 * j), up(3 * j + 1), up(3 * j + 2)},
        []int{up(3 * j) ⊕ 1, up(3 * j + 1) ⊕ 1, up(3 * j + 2) ⊕ 1})
}
```

This code is used in section 22.

24. ⟨ Check answers 204(a), 207, and 208 24 ⟩ ≡

```
fmt.Println("204(a). Tovey's clauses")
t := tovey(parseCNF(2, "1 1 2'"))
claim(t.String() ≡ "1 2 3, 1' 2, 2' 1, 3 3'",
    "(x1 v x1 v x2') becomes %s, as printed", t)
⟨ Check answer 207 25 ⟩
⟨ Check answer 208 27 ⟩
```

This code is used in section 3.

25. Answer 207 says that $C(x, y, z; a, b, c)$ "resolves to the single clause xyz." The program checks the stronger fact that makes the construction work: C is satisfiable exactly for the x, y, z that satisfy xyz. Then the twenty clauses need three properties, and the note's sixteen clauses two. The note also reports that no set of twelve clauses uses every variable four times and is unsatisfiable; that is a search of another size, and it is not checked here.

⟨ Check answer 207 25 ⟩ ≡

```
fmt.Println("207. twenty clauses, every literal twice")
g := cnf{n: 6, cl: gadget(2, 4, 6, 8, 10, 12)}
proj := map[uint64]bool{ }
for _, a := range models(g) {
    proj[a & 14] = true
}
claim(len(proj) ≡ 7 ∧ ¬proj[0], "C(x,y,z;a,b,c) can be satisfied exactly " +
    "when x v y v z is true")
a207 := answer207( )
occ := occurrences(a207)
claim(len(a207.cl) ≡ 20 ∧ slices.Min(occ[2:]) ≡ 2 ∧ slices.Max(occ[2:]) ≡ 2,
    "answer 207 has 20 clauses, and each of its 30 literals occurs twice")
claim(len(models(a207)) ≡ 0, "they are unsatisfiable")
⟨ Check that every clause of answer 207 is necessary 26 ⟩
it := iwamaTakaki( )
occ = occurrences(it)
fours := true
for v := 1; v ≤ 12; v++ {
    fours = fours ∧ occ[2 * v] + occ[2 * v + 1] ≡ 4
}
claim(fours ∧ len(models(it)) ≡ 0, "the note's 16 clauses use each of 12 " +
    "variables four times, and are unsatisfiable")
```

This code is used in section 24.

26. Answer 208 relies on something the text leaves implicit: when one clause is left out, the other nineteen are satisfiable, and every solution makes the three literals of the missing clause false. Otherwise the cloned literals would change the binary clauses they are put into.

⟨ Check that every clause of answer 207 is necessary 26 ⟩ ≡

```
okDrop := true
for d := range a207.cl {
    rest := cnf{n: 15, cl: slices.Delete(slices.Clone(a207.cl), d, d + 1)}
    ms := models(rest)
    okDrop = okDrop ∧ len(ms) > 0
    for _, a := range ms {
        okDrop = okDrop ∧ ¬slices.ContainsFunc(a207.cl[d],
            func(l int) bool { return holds(l, a) })
    }
}
claim(okDrop, "leaving out any one clause makes them satisfiable, and every " +
    "solution then falsifies that clause")
```

This code is used in section 25.

27. Random 3SAT problems with up to six variables and twelve clauses, repeated literals allowed, are transformed with every choice of the omitted clause and both parities. The original is decided by brute force, the result, with $21m$ variables, by the package.

⟨ Check answer 208 27 ⟩ ≡

```
fmt.Println("208. every literal exactly twice")
r := rng(208)
var n208, badShape, badSat, badPad int
for trial := 0; trial < reps; trial++ {
    f := cnf{n: 3 + r.IntN(4)}
    for k, m := 0, 1 + r.IntN(12); k < m; k++ {
        f.cl = append(f.cl, []int{2 * (1 + r.IntN(f.n)) + r.IntN(2),
            2 * (1 + r.IntN(f.n)) + r.IntN(2), 2 * (1 + r.IntN(f.n)) + r.IntN(2)})
    }
    m := len(f.cl)
    g, before := clones208(f, trial % 20, trial % 2 ≡ 0)
    n208++
    ⟨ Check the shape of g 28 ⟩
    if (len(models(f)) > 0) ≠ solveCNF(g) {
        badSat++
    }
    ⟨ Check that the padding can always be satisfied 29 ⟩
}
claim(badShape ≡ 0, "%d transformations: 23m clauses before padding, 28m " +
    "after, 21m variables, every literal twice; %d exceptions", n208, badShape)
claim(badSat ≡ 0, "satisfiable exactly when the original is: %d exceptions",
    badSat)
claim(badPad ≡ 0, "the padding is satisfied whenever all u_i are equal: " +
    "%d exceptions", badPad)
```

This code is used in section 24.

28. Before padding, every literal should occur twice except the $\bar{X}_i$, which occur once, and the u_i should not occur at all.

⟨ Check the shape of g 28 ⟩ ≡

```
occB := occurrences(cnf{n: g.n, cl: g.cl[:before]})
occA := occurrences(g)
good := before ≡ 23 * m ∧ len(g.cl) ≡ 28 * m ∧ g.n ≡ 21 * m
for v := 1; v ≤ g.n; v++ {
    wantPos, wantNeg := 2, 2
    if v ≤ 3 * m {
        wantNeg = 1
    } else if v > 18 * m {
        wantPos, wantNeg = 0, 0
    }
    good = good ∧ occB[2 * v] ≡ wantPos ∧ occB[2 * v + 1] ≡ wantNeg ∧
        occA[2 * v] ≡ 2 ∧ occA[2 * v + 1] ≡ 2
}
if ¬good {
    badShape++
}
```

This code is used in section 27.

29. ⟨ Check that the padding can always be satisfied 29 ⟩ ≡

```
for _, same := range []uint64 { 0, ⊕uint64(0) }{
    for i := before; i < len(g.cl); i++ {
        ok := false
        for _, l := range g.cl[i] {
            v := l ≫ 1
            val := r.IntN(2) ≡ 1
            if v > 18 ∗ m {
                val = same ≠ 0
            }
            ok = ok ∨ val ≡ (l & 1 ≡ 0)
        }
        if ¬ok {
            badPad++
        }
    }
}
```

This code is used in section 27.

30. The grid: answer 211. Given m clauses in n variables with $3m = 4n$, the answer uses $N = 8n$ colors named jk and $\overline{jk}$, where k is one of the four clauses that contain $\pm x_j$, and a permutation σ of the colors made of 4-cycles, one for each variable, with $(jk)\sigma = jk'$ and $(\overline{jk})\sigma = \overline{(jk)\sigma}$. The $4n$ vertices jk of K_N have the lists

$$L(jk,1) = \{jk, \overline{jk}\}, \quad L(jk,2) = \{jk, (jk)\sigma\}, \quad L(jk,3) = \{\overline{jk}, (jk)\sigma\},$$

and the $3m$ vertices a_k, b_k, c_k, for a clause such as $x_2 \vee \bar{x}_5 \vee x_6$, have

$$L(a_k,1) = \{2k, \overline{5k}, 6k\}, \quad L(b_k,1) = L(c_k,1) = \{2k, \overline{2k}, 5k, \overline{5k}, 6k, \overline{6k}\};$$
$$L(a_k,2) = \{\overline{(2k)\sigma}\}, \quad L(b_k,2) = \{\overline{(5k)\sigma}\}, \quad L(c_k,2) = \{\overline{(6k)\sigma}\};$$
$$L(a_k,3) = \{\overline{(2k)\sigma^2}, (2k)\sigma\}, \quad L(b_k,3) = \{\overline{(5k)\sigma^2}, (5k)\sigma\}, \quad L(c_k,3) = \{\overline{(6k)\sigma^2}, (6k)\sigma\}.$$

The program names things by occurrence rather than by clause. Occurrence $q = 3k+t$ is literal t of clause k, both counted from 0; its colors are $2q$ for jk and $2q+1$ for $\overline{jk}$, its vertex is q, and the vertex a_k, b_k, or c_k that belongs to it is $3m + q$. So $N = 6m$, which is $8n$ when $3m = 4n$. The permutation *sigma* acts on occurrences. The lists are indexed first by the column $1, 2, 3$ of the book, counted from 0, then by vertex, which is the shape $K_M \,\square\, K_N$ of exercise 212(c) with $M = 3$.

⟨ Types 8 ⟩ +≡

```
type lists [][][]int
```

31. ⟨ Functions 6 ⟩ +≡

```
func grid211(f cnf, sigma []int) lists {
    m := len(f.cl)
    N := 6 * m
    L := lists{make([][]int, N), make([][]int, N), make([][]int, N)}
    for q := 0; q < 3 * m; q++ {
        s := sigma[q]
        L[0][q] = []int{2 * q, 2 * q + 1}
        L[1][q] = []int{2 * q, 2 * s}
        L[2][q] = []int{2 * q + 1, 2 * s}
        ⟨ Give the vertex 3m + q its three lists 32 ⟩
    }
    for I := range L {
        for J := range L[I] {
            slices.Sort(L[I][J])
        }
    }
    return L
}
```

32. ⟨ Give the vertex $3m + q$ its three lists 32 ⟩ ≡

```
w, k := 3 * m + q, q/3
if q % 3 ≡ 0 {
    for t, l := range f.cl[k] {
        L[0][w] = append(L[0][w], 2 * (3 * k + t) + l & 1)
    }
} else {
    for t := 0; t < 3; t++ {
        L[0][w] = append(L[0][w], 2 * (3 * k + t), 2 * (3 * k + t) + 1)
    }
}
L[1][w] = []int{2 * s + 1}
L[2][w] = []int{2 * sigma[s] + 1, 2 * s}
```

This code is used in section 31.

33. The answer asks for 4-cycles that "involve the same variable" and says nothing more about their order. *cycles* makes one random cycle through the occurrences of each variable. If *inside* is false it keeps shuffling until no step of the cycle stays inside a clause, which is possible whenever a variable occurs four times and no clause holds it more than twice.

⟨ Functions 6 ⟩ +≡

```
func cycles(f cnf, r *rand.Rand, inside bool) []int {
    m := len(f.cl)
    occ := make([][]int, f.n + 1)
    for q := 0; q < 3 * m; q++ {
        occ[f.cl[q/3][q % 3] ≫ 1] = append(occ[f.cl[q/3][q % 3] ≫ 1], q)
    }
    sigma := make([]int, 3 * m)
    for _, o := range occ {
        for try := 0; try < 1000; try++ {
            r.Shuffle(len(o), func(i, j int) { o[i], o[j] = o[j], o[i] })
            stays := false
            for i, q := range o {
                stays = stays ∨ len(o) > 1 ∧ q/3 ≡ o[(i + 1) % len(o)]/3
            }
            if inside ∨ ¬stays {
                break
            }
        }
        for i, q := range o {
            sigma[q] = o[(i + 1) % len(o)]
        }
    }
    return sigma
}
```

34. A list coloring of $K_M \,\square\, K_N$ is an exact cover: one option (I, J, K) per cell, the MN cells primary, and the pairs row–color and column–color secondary. The colors are $0, \ldots, N-1$. The function also returns the triples, to decode solutions.

⟨ Functions 6 ⟩ +≡

```
func colorCover(L lists, N int)(*xcover, [][3]int) {
    M := len(L)
    x := newCover(M * N, 2 * M * N + N * N)
    var triples [][3]int
    for I := 0; I < M; I++ {
        for J := 0; J < N; J++ {
            for _, K := range L[I][J] {
                x.add(I * N + J, M * N + I * N + K, 2 * M * N + J * N + K)
                triples = append(triples, [3]int{I, J, K})
            }
        }
    }
    return x, triples
}
func colorings(L lists, N int, visit func(col [][]int) bool) {
    x, triples := colorCover(L, N)
    x.search(func(sol []int) bool {
        col := make([][]int, len(L))
        for I := range col {
            col[I] = make([]int, N)
        }
        for _, o := range sol {
            col[triples[o][0]][triples[o][1]] = triples[o][2]
        }
        return visit(col)
    })
}
```

35. The same coloring problem for the package: one variable per option, at least one per cell, at most one per row–color and column–color pair. A cell may come out with more than one true variable; the program reads the first, and the result is still a coloring, since the pairs constrain every true variable.

⟨ Functions 6 ⟩ +≡

```
func colorSAT(L lists, N int)([][]int, bool) {
    s := sat.New()
    M := len(L)
    rows, cols := map[[2]int][]sat.Lit{}, map[[2]int][]sat.Lit{}
    lit := make([][][]sat.Lit, M)
    for I := 0; I < M; I++ {
        lit[I] = make([][]sat.Lit, N)
        for J := 0; J < N; J++ {
            for _, K := range L[I][J] {
                l := s.NewVar()
                lit[I][J] = append(lit[I][J], l)
                rows[[2]int{I, K}] = append(rows[[2]int{I, K}], l)
                cols[[2]int{J, K}] = append(cols[[2]int{J, K}], l)
            }
            s.AddClause(lit[I][J]...)
        }
    }
    for _, group := range []map[[2]int][]sat.Lit { rows, cols }{
        for _, ls := range group {
            atMostOne(s, ls)
        }
    }
    if st, _ := s.Solve(context.Background()); st ≠ sat.Sat {
        return Λ, false
    }
    ⟨ Read the coloring off the model 36 ⟩
}
func atMostOne(s *sat.Solver, ls []sat.Lit) {
    for i := range ls {
        for j := i + 1; j < len(ls); j++ {
            s.AddClause(ls[i].Not(), ls[j].Not())
        }
    }
}
```

36. ⟨ Read the coloring off the model 36 ⟩ ≡

```
col := make([][]int, M)
for I := 0; I < M; I++ {
    col[I] = make([]int, N)
    for J := 0; J < N; J++ {
        for t, l := range lit[I][J] {
            if s.Value(l) {
                col[I][J] = L[I][J][t]
                break
            }
        }
    }
}
return col, true
```

This code is used in section 35.

37. Whatever produced a coloring, it is checked against the definition.

⟨ Functions 6 ⟩ +≡

```
func validColoring(L lists, N int, col [][]int) bool {
    for I := range L {
        seen := make([]bool, N)
        for J := 0; J < N; J++ {
            K := col[I][J]
            if ¬slices.Contains(L[I][J], K) ∨ seen[K] {
                return false
            }
            seen[K] = true
        }
    }
    for J := 0; J < N; J++ {
        seen := make([]bool, N)
        for I := range L {
            if seen[col[I][J]] {
                return false
            }
            seen[col[I][J]] = true
        }
    }
    return true
}
```

38. In column 2 the vertices a_k, b_k, c_k take all the colors $\overline{jk}$, so every vertex jk takes jk or $(jk)\sigma$, and all four of one variable make the same choice. Choosing jk forces $\overline{jk}$ in column 1, which frees the color jk there; so jk in column 2 means x_j is true, and *assignment* reads the solution that way.

⟨ Functions 6 ⟩ +≡

```
func assignment(f cnf, col [][]int) uint64 {
    var a uint64
    for q := 0; q < 3 * len(f.cl); q++ {
        if col[1][q] ≡ 2 * q {
            a |= 1 ≪ (f.cl[q/3][q % 3] ≫ 1)
        }
    }
    return a
}
```

39. The other direction, done by hand. If x_j is true, vertex jk gets $\overline{jk}$, jk, $(jk)\sigma$ in its three columns; if false, jk, $(jk)\sigma$, $\overline{jk}$. The colors of clause k left free in column 1 are then jk or $\overline{jk}$ for its three occurrences, exactly the literal's own color when the literal is true; a_k takes one of those and b_k, c_k take the other two. Columns 2 and 3 of a_k, b_k, c_k are forced: $\overline{(jk)\sigma}$, and $\overline{(jk)\sigma^2}$ or $(jk)\sigma$ as x_j is true or false.

⟨ Functions 6 ⟩ +≡

```
func colorFromModel(f cnf, sigma []int, a uint64) [][]int {
    m := len(f.cl)
    col := [][]int{make([]int, 6 * m), make([]int, 6 * m), make([]int, 6 * m)}
    value := func(q int) bool { return a ≫ (f.cl[q/3][q % 3] ≫ 1) & 1 ≡ 1 }
    for q := 0; q < 3 * m; q++ {
        s, w := sigma[q], 3 * m + q
        if value(q) {
            col[0][q], col[1][q], col[2][q] = 2 * q + 1, 2 * q, 2 * s
            col[2][w] = 2 * sigma[s] + 1
        } else {
            col[0][q], col[1][q], col[2][q] = 2 * q, 2 * s, 2 * q + 1
            col[2][w] = 2 * s
        }
        col[1][w] = 2 * s + 1
    }
    ⟨ Hand out the free colors of column 1, clause by clause 40 ⟩
    return col
}
```

40. ⟨ Hand out the free colors of column 1, clause by clause 40 ⟩ ≡

```
for k := 0; k < m; k++ {
    var free []int
    first := -1
    for t := 0; t < 3; t++ {
        q := 3 * k + t
        c := 2 * q
        if ¬value(q) {
            c++
        }
        if first < 0 ∧ holds(f.cl[k][t], a) {
            first = len(free)
        }
        free = append(free, c)
    }
    if first < 0 {
        first = 0
    }
    free[0], free[first] = free[first], free[0]
    copy(col[0][3 * m + 3 * k:], free)
}
```

This code is used in section 39.

41. Random problems in which every variable occurs exactly four times. With *distinct* the three literals of each clause belong to different variables, as they do when k can name "one of the four clauses." Otherwise only repeated literals are excluded.

⟨ Functions 6 ⟩ +≡

```
func fourTimes(r *rand.Rand, n int, distinct bool) cnf {
    for {
        var ls []int
        for v := 1; v ≤ n; v++ {
            for t := 0; t < 4; t++ {
                ls = append(ls, 2 * v + r.IntN(2))
            }
        }
        r.Shuffle(len(ls), func(i, j int) { ls[i], ls[j] = ls[j], ls[i] })
        f, ok := cnf{n: n}, true
        for i := 0; i < len(ls); i += 3 {
            c := ls[i:i + 3]
            same := c[0] ⊕ c[1] < 2 ∨ c[0] ⊕ c[2] < 2 ∨ c[1] ⊕ c[2] < 2
            ok = ok ∧ c[0] ≠ c[1] ∧ c[0] ≠ c[2] ∧ c[1] ≠ c[2] ∧
                (¬distinct ∨ ¬same)
            f.cl = append(f.cl, c)
        }
        if ok {
            return f
        }
    }
}
```

42. ⟨ Check answer 211 42 ⟩ ≡

```
fmt.Println("211. 3SAT to list coloring of K_N x K_3")
⟨ Count the colorings of small grids 43 ⟩
⟨ Decide larger grids with the package 44 ⟩
⟨ Check the example in parentheses 45 ⟩
⟨ Look at clauses that hold a variable twice 46 ⟩
```

This code is used in section 3.

43. The argument above does more than decide colorability; it counts. Column 1 of clause k can be completed in $2t_k$ ways, where t_k is the number of true literals of the clause: a_k picks one of the t_k and b_k, c_k share the other two. Everything else is forced. So the number of colorings should be $\sum_x \prod_k 2t_k(x)$ over the solutions x, and the program compares that with a count by exact cover, for *reps* problems, one in five of them on six variables and the rest on three; a grid for six variables can have close to a million colorings, which the ZDD engine counts in a fraction of a second. The program also colors the grid by hand from every solution.

⟨ Count the colorings of small grids 43 ⟩ ≡

```
r := rng(211)
var nSmall, nSix, badCount, badHand int
for trial := 0; trial < reps; trial++ {
    n := 3
    if trial % 5 ≡ 4 {
        n = 6
    }
    f := fourTimes(r, n, true)
    sigma := cycles(f, r, false)
    L := grid211(f, sigma)
    N := 6 * len(f.cl)
    want := 0
    for _, a := range models(f) {
        prod := 1
        for _, c := range f.cl {
            prod *= 2 * len(slices.DeleteFunc(slices.Clone(c),
                func(l int) bool { return ¬holds(l, a) }))
        }
        want += prod
        if col := colorFromModel(f, sigma, a); ¬validColoring(L, N, col) ∨
            assignment(f, col) ≠ a {
            badHand++
        }
    }
    x, _ := colorCover(L, N)
    nSmall++
    if n ≡ 6 {
        nSix++
    }
    if x.total() ≠ want {
        badCount++
    }
}
claim(badCount ≡ 0, "%d grids, %d of them for 6 variables: the colorings " +
    "number sum over solutions of prod 2t_k, with %d exceptions", nSmall, nSix,
    badCount)
claim(badHand ≡ 0, "every solution colors its grid by hand, and the coloring " +
    "gives the solution back: %d exceptions", badHand)
```

This code is used in section 42.

44. The unsatisfiable problems cannot be small: by the note in answer 207, which this program does not check, they need at least sixteen clauses. So the package colors, or fails to color, the grids of the two unsatisfiable problems at hand, the grid of answer 208 applied to the eight clauses on three variables, and random satisfiable problems on 30 to 60 variables. A coloring it finds is checked, and read back as an assignment that must satisfy the clauses.

⟨ Decide larger grids with the package 44 ⟩ ≡

```
cube := cnf{n: 3}
for a := 0; a < 8; a++ {
    cube.cl = append(cube.cl, []int{2 + a & 1, 4 + a >> 1 & 1, 6 + a >> 2 & 1})
}
cube208, _ := clones208(cube, 3, true)
for _, g := range []struct {
    name string
    f cnf
}{{"answer 207", answer207()}, {"the note's 16 clauses", iwamaTakaki()},
    {"answer 208 of all 8 clauses on 3 variables", cube208}}{
        _, ok := colorSAT(grid211(g.f, cycles(g.f, r, false)), 6 * len(g.f.cl))
        claim(¬ok ∧ ¬solveCNF(g.f), "%s: unsatisfiable, and the grid for N = %d " +
            "cannot be colored", g.name, 6 * len(g.f.cl))
    }
var nBig, badBig int
for trial := 0; trial < reps/10; trial++ {
    f := fourTimes(r, 30 + 3 * (trial % 11), true)
    L := grid211(f, cycles(f, r, false))
    col, ok := colorSAT(L, 6 * len(f.cl))
    nBig++
    if ok ≠ solveCNF(f) ∨ ok ∧ (¬validColoring(L, 6 * len(f.cl), col) ∨
        ¬satisfies(f, assignment(f, col))) {
        badBig++
    }
}
claim(badBig ≡ 0, "%d problems on 30 to 60 variables: colorings found, " +
    "checked, and decoded to solutions; %d exceptions", nBig, badBig)
```

This code is used in section 42.

45. The answer ends with an example: "$(jk, 1)$ is colored $jk \iff ((jk)\sigma, 1)$ is colored $(jk)\sigma \iff (a_k, 1)$ is not colored jk." The first equivalence and the implication from left to right hold, since a used color is not free. The implication from right to left does not: when x_j is true the color jk is free in column 1, but a clause with more than one true literal may give it to b_k or c_k instead of a_k. The program counts, over the colorings of small grids, the cases where x_j occurs positively in clause k, so that jk is on a_k's list, and the equivalence still fails.

⟨ Check the example in parentheses 45 ⟩ ≡

```
var nCol, badFirst, badRight, badLeft int
for trial := 0; trial < reps/4; trial++ {
    f := fourTimes(r, 3, true)
    sigma := cycles(f, r, false)
    m := len(f.cl)
    colorings(grid211(f, sigma), 6 * m, func(col [][]int) bool {
        nCol++
        for q := 0; q < 3 * m; q++ {
            left := col[0][q] ≡ 2 * q
            if left ≠ (col[0][sigma[q]] ≡ 2 * sigma[q]) {
                badFirst++
            }
            if right := col[0][3 * m + 3 * (q/3)] ≠ 2 * q; left ∧ ¬right {
                badRight++
            } else if ¬left ∧ right ∧ f.cl[q/3][q % 3] & 1 ≡ 0 {
                badLeft++
            }
        }
        return true
    })
}
claim(badFirst ≡ 0 ∧ badRight ≡ 0, "%d colorings: the first equivalence " +
    "and the last implication from left to right always hold", nCol)
claim(badLeft > 0, "(a_k,1) is not colored jk, yet (jk,1) is not colored jk " +
    "either, in %d cases with x_j positive in clause k", badLeft)
```

This code is used in section 42.

46. Clauses with a variable twice. The answer's names jk presuppose that the four occurrences of x_j lie in four different clauses. Answer 208 does not promise that. When a variable of the original problem occurs only once, answer 204(a) gives it the clause $(X_i \vee \bar{X}_i)$, with signs in one order or the other, and answer 208 puts a cloned literal into it; the book shows the degenerate case itself. The resulting clause has four occurrences of X_i in three clauses, and "the four clauses that contain $\pm x_j$" are three.

Naming by occurrence repairs the names, but σ still matters. Take a clause $k = (x \vee \bar{x} \vee l)$, let q and q' be the occurrences of x and $\bar{x}$, so that a_k belongs to q, and suppose $q\sigma = q'$. Then a_k has the color $\overline{q'}$ in column 2. If x is false, the colors of clause k left free in column 1 are $\overline{q}$, $\overline{q'}$, and one for l; the list of a_k offers q, $\overline{q'}$, and the color of l; and $\overline{q'}$ is taken. So a_k needs l true, the clause behaves like $(x \vee l)$, and a satisfiable problem can give an uncolorable grid.

That cannot happen when no step of σ stays inside a clause. Only a_k's choice in column 1 can be forced, since b_k and c_k take any color. Column 2 of a_k then belongs to another clause; and column 3 belongs to clause k only when $q\sigma^2$ comes back to it, in which case it is $\overline{q\sigma^2}$ for a true variable, a color that is not free in column 1.

⟨ Look at clauses that hold a variable twice 46 ⟩ ≡

⟨ Check that answer 208 produces such clauses 47 ⟩
⟨ Show a problem where a step inside a clause breaks the reduction 49 ⟩
⟨ Check that cycles which leave the clause always work 50 ⟩

This code is used in section 42.

47. In answer 208 the harm cannot happen: the clause is $(\bar{X} \vee X \vee l)$ or $(X \vee \bar{X} \vee l)$, and in the second case X stands for a literal that occurs once in the original problem, so it can always be made true. The program transforms random problems in which x_4 occurs once, colors each grid with four random choices of cycles, which are free to step inside clauses, and compares with the original.

⟨ Check that answer 208 produces such clauses 47 ⟩ ≡

```
var n208, withPair, nInside, badPair int
for trial := 0; trial < reps/10; trial++ {
    f := cnf{n: 4, cl: [][]int{{2, 4, 8 + r.IntN(2)}}}
    for k := 0; k < 1 + trial % 4; k++ {
        f.cl = append(f.cl, []int{2 * (1 + r.IntN(3)) + r.IntN(2),
            2 * (1 + r.IntN(3)) + r.IntN(2), 2 * (1 + r.IntN(3)) + r.IntN(2)})
    }
    g, _ := clones208(f, trial % 20, true)
    n208++
    for _, c := range g.cl {
        if c[0] ⊕ c[1] ≡ 1 {
            withPair++
            break
        }
    }
    ⟨ Color the grid of g with several cycles 48 ⟩
}
claim(withPair ≡ n208, "all␣%d␣transformations␣of␣problems␣in␣which␣x4␣" +
    "occurs␣once␣contain␣a␣clause␣(X␣v␣X'␣v␣l)", n208)
claim(badPair ≡ 0, "%d␣of␣their␣grids,␣%d␣with␣steps␣inside␣a␣clause,␣are␣" +
    "colorable␣exactly␣when␣the␣problem␣is␣satisfiable:␣%d␣exceptions",
    4 * n208, nInside, badPair)
```

This code is used in section 46.

48. ⟨ Color the grid of g with several cycles 48 ⟩ ≡

```
for try := 0; try < 4; try++ {
    sigma := cycles(g, r, true)
    for q := range sigma {
        if q ≠ sigma[q] ∧ q/3 ≡ sigma[q]/3 {
            nInside++
            break
        }
    }
    _, ok := colorSAT(grid211(g, sigma), 6 * len(g.cl))
    if ok ≠ (len(models(f)) > 0) {
        badPair++
    }
}
```

This code is used in section 47.

49. Outside answer 208, it does happen. The following twelve variables and sixteen clauses use every variable exactly four times, with three different literals in each clause, and every one of their 90 solutions makes x_1 and x_2 false. The first clause is $(x_1 \vee \bar{x}_1 \vee x_2)$. A local search found it.

⟨ Show a problem where a step inside a clause breaks the reduction 49 ⟩ ≡

```
bad := parseCNF(12, "1␣1'␣2,␣1'␣9␣5',␣8'␣10'␣7',␣6␣12␣4,␣6'␣9'␣10,␣6'␣8␣12,␣" +
    "12␣4'␣6,␣2'␣11␣9,␣5'␣11'␣3,␣2'␣11'␣5,␣9'␣3␣11,␣4'␣7'␣10,␣5␣1'␣2,␣" +
    "8␣12'␣3',␣7␣8'␣3',␣4␣10␣7'")
occ := occurrences(bad)
four := true
for v := 1; v ≤ 12; v++ {
    four = four ∧ occ[2 * v] + occ[2 * v + 1] ≡ 4
}
ms := models(bad)
forced := true
for _, a := range ms {
    forced = forced ∧ a & 6 ≡ 0
}
claim(four ∧ len(ms) ≡ 90 ∧ forced, "the␣problem␣has␣90␣solutions,␣all␣" +
    "with␣x1␣and␣x2␣false")
inSigma := cycles(bad, r, true)
for inSigma[0] ≠ 1 {
    inSigma = cycles(bad, r, true)
}
x, _ := colorCover(grid211(bad, inSigma), 96)
_, okIn := colorSAT(grid211(bad, inSigma), 96)
claim(¬x.exists() ∧ ¬okIn, "with␣sigma␣taking␣x1␣to␣x1'␣in␣the␣first␣" +
    "clause,␣the␣grid␣cannot␣be␣colored")
_, okOut := colorSAT(grid211(bad, cycles(bad, r, false)), 96)
claim(okOut, "with␣cycles␣that␣leave␣every␣clause,␣it␣can")
```

This code is used in section 46.

50. ⟨ Check that cycles which leave the clause always work 50 ⟩ ≡

```
var nOut, badOut int
for trial := 0; trial < reps; trial++ {
    f := fourTimes(r, 3 + 3 * (trial % 2), false)
    sigma := cycles(f, r, false)
    sepa := true
    for q := range sigma {
        sepa = sepa ∧ q/3 ≠ sigma[q]/3
    }
    if ¬sepa {
        continue
    }
    x, _ := colorCover(grid211(f, sigma), 6 * len(f.cl))
    nOut++
    if x.exists() ≠ (len(models(f)) > 0) {
        badOut++
    }
}
claim(badOut ≡ 0, "%d random problems allowing (x v x' v l), with cycles " +
    "that leave every clause: %d exceptions", nOut, badOut)
```

This code is used in section 46.

51. The tensor: answer 212(a). The answer is one line: "Let $x_{ijk} = 1$ if and only if $X_{ij} = k$." Under that correspondence $x_{ij*} = p_{ij}$ says that a cell is filled exactly when $p_{ij} = 1$, and $x_{i*k} \le r_{ik}$, $x_{*jk} \le c_{jk}$ say that the symbols are allowed and not repeated. But the tensor asks for equality, $x_{i*k} = r_{ik}$ and $x_{*jk} = c_{jk}$, which also says that row i contains *every* k with $r_{ik} = 1$, and column j every k with $c_{jk} = 1$. The array problem, as the exercise words it, does not demand that. With $n = 1$, $r = c = (1)$, and $p = (0)$ the blank array solves it, while the tensor must have $x_{11*} = 0$ and $x_{1*1} = 1$.

So the two problems agree only if r and c are read as saying which symbols *must* occur. That is the reading part (b) needs, because under the other one the condition $r_{i*} = p_{i*}$ is not necessary, as the same example shows. It is also the reading of the exact cover in the answer's note, whose items Rik and Cjk are primary.

A partial latin square is kept sparse: the list of cells with $p_{ij} = 1$, and for each index the sorted lists of k with $r_{ik} = 1$ and with $c_{ik} = 1$.

⟨ Types 8 ⟩ +≡

```
type square struct {
    n int
    p [][2]int
    r, c [][]int
}
```

52. The exact cover for a square. With *exact* the pairs Rik and Cjk are primary, which is the tensor; without it they are secondary, which is the array as worded. The triples come back for decoding.

⟨ Functions 6 ⟩ +≡

```
func squareCover(q square, exact bool)(*xcover, [][3]int) {
    id := map[[3]int]int{}
    for _, cell := range q.p {
        id[[3]int{0, cell[0], cell[1]}] = len(id)
    }
    primary := len(id)
    for i := 0; i < q.n; i++ {
        for _, k := range q.r[i] {
            id[[3]int{1, i, k}] = len(id)
        }
        for _, k := range q.c[i] {
            id[[3]int{2, i, k}] = len(id)
        }
    }
    if exact {
        primary = len(id)
    }
    ⟨ Make the options of the square 53 ⟩
    return x, triples
}
```

53. ⟨ Make the options of the square 53 ⟩ ≡

```
x := newCover(primary, len(id))
var triples [][3]int
for _, cell := range q.p {
    i, j := cell[0], cell[1]
    for _, k := range q.r[i] {
        if _, ok := slices.BinarySearch(q.c[j], k); ok {
            x.add(id[[3]int{0, i, j}], id[[3]int{1, i, k}], id[[3]int{2, j, k}])
            triples = append(triples, [3]int{i, j, k})
        }
    }
}
```

This code is used in section 52.

54. Small squares are given by three bit patterns, bit $in + j$ for entry (i, j).

⟨ Functions 6 ⟩ +≡

```
func fromBits(n int, p, r, c uint64) square {
    q := square{n: n, r: make([][]int, n), c: make([][]int, n)}
    for i := 0; i < n; i++ {
        for j := 0; j < n; j++ {
            if p >> (i * n + j) & 1 == 1 {
                q.p = append(q.p, [2]int{i, j})
            }
            if r >> (i * n + j) & 1 == 1 {
                q.r[i] = append(q.r[i], j)
            }
            if c >> (i * n + j) & 1 == 1 {
                q.c[i] = append(q.c[i], j)
            }
        }
    }
    return q
}
```

55. The condition of part (b): $c_{*k} = r_{*k}$, $c_{j*} = p_{*j}$, and $r_{i*} = p_{i*}$.

⟨ Functions 6 ⟩ +≡

```
func balanced(q square) bool {
    rk, ck := make([]int, q.n), make([]int, q.n)
    pi, pj := make([]int, q.n), make([]int, q.n)
    for i := 0; i < q.n; i++ {
        for _, k := range q.r[i] {
            rk[k]++
        }
        for _, k := range q.c[i] {
            ck[k]++
        }
    }
    for _, cell := range q.p {
        pi[cell[0]]++
        pj[cell[1]]++
    }
    for a := 0; a < q.n; a++ {
        if ck[a] ≠ rk[a] ∨ len(q.c[a]) ≠ pj[a] ∨ len(q.r[a]) ≠ pi[a] {
            return false
        }
    }
    return true
}
```

56. All 2^{3n^2} instances with $n \le 2$ are tried under both readings. Under the tensor reading the problem is symmetrical, and the program checks one of the symmetries that makes it so: exchanging the roles of j and k turns (p, r, c) into (r, p, c^T) and keeps the number of solutions.

```
⟨ Check answer 212(a) 56 ⟩ ≡
    fmt.Println("212(a). the array and the tensor")
    one := fromBits(1, 0, 1, 1)
    x1, _ := squareCover(one, false)
    x2, _ := squareCover(one, true)
    claim(x1.total() ≡ 1 ∧ x2.total() ≡ 0 ∧ ¬balanced(one),
        "n = 1, r = c = 1, p = 0: the blank array solves it, the tensor has no " +
            "solution, and r_1* = p_1* fails")
    for n := 1; n ≤ 2; n++ {
        nb := uint(n * n)
        var total, differ, badSym, badNec int
        for bits := uint64(0); bits < 1 ≪ (3 * nb); bits++ {
            p, rr, c := bits & (1 ≪ nb − 1), bits ≫ nb & (1 ≪ nb − 1), bits ≫ (2 * nb)
            q := fromBits(n, p, rr, c)
            xa, _ := squareCover(q, false)
            xt, _ := squareCover(q, true)
            total++
            ⟨ Compare the readings, the symmetry, and the necessary condition 57 ⟩
        }
        claim(badSym ≡ 0 ∧ badNec ≡ 0, "n = %d: %d instances; the readings " +
            "disagree on %d; the tensor keeps its count under j <-> k, and " +
            "satisfies the condition of (b) whenever it is solvable", n, total,
            differ)
    }
```

This code is used in section 3.

57.

```
⟨ Compare the readings, the symmetry, and the necessary condition 57 ⟩ ≡
    tensor := xt.total()
    if xa.exists() ≠ (tensor > 0) {
        differ++
    }
    var ct uint64
    for i := 0; i < n; i++ {
        for j := 0; j < n; j++ {
            ct |= c ≫ (i * n + j) & 1 ≪ (j * n + i)
        }
    }
    if xs, _ := squareCover(fromBits(n, rr, p, ct), true); xs.total() ≠ tensor {
        badSym++
    }
    if tensor > 0 ∧ ¬balanced(q) {
        badNec++
    }
```

This code is used in section 56.

58. The small example: answer 212(b). "$c_{31} = c_{32} = r_{13} = r_{14} = 0$ forces $x_{13*} = 0 \neq p_{13}$ when $r = c = \binom{1100}{0110}\binom{0011}{1001}$" in the book's 4×4 layout, and p has rows 1010, 1100, 0101, 0011. The program checks that the condition holds, that each of the four entries is 0 and $p_{13} = 1$, that the cell has no possible symbol, and that there is no solution. Then it finds how small an example can be, by trying every instance for $n \leq 3$ that meets the condition.

⟨ Check answer 212(b) 58 ⟩ ≡

```
fmt.Println("212(b). the condition is not sufficient")
rc := uint64(ᵇ1001 1100 0110 0011)
pb := uint64(ᵇ1100 1010 0011 0101)
ex := fromBits(4, pb, rc, rc)
xb, _ := squareCover(ex, true)
_, c31 := slices.BinarySearch(ex.c[2], 0)
_, c32 := slices.BinarySearch(ex.c[2], 1)
entries := ¬c31 ∧ ¬c32 ∧ ¬slices.Contains(ex.r[0], 2) ∧
    ¬slices.Contains(ex.r[0], 3) ∧ slices.Contains(ex.p, [2]int{0, 2})
claim(balanced(ex) ∧ entries ∧ ¬xb.exists( ),
    "the answer's example meets the condition, p13 = 1 has no symbol, and " +
        "there is no solution")
⟨ Find the smallest examples 59 ⟩
```

This code is used in section 3.

59. The bit patterns are read from the right: bit $4i+j$ is entry (i,j), counted from 0, so the first row 1100 of r is the pattern 0011 at the low end.

For $n=3$ there are 2^{27} instances, but r and c must have equal column sums, and p must then have row sums r_{i*} and column sums c_{j*}; the loops skip the rest early.

```
⟨ Find the smallest examples 59 ⟩ ≡
    unsolvable := make([]int, 4)
    for n := 1; n ≤ 3; n++ {
        nb := uint(n * n)
        var meet int
        var first square
        for rr := uint64(0); rr < 1 ≪ nb; rr++ {
            for c := uint64(0); c < 1 ≪ nb; c++ {
                ⟨ Skip unless rr and c have the same column sums 60 ⟩
                for p := uint64(0); p < 1 ≪ nb; p++ {
                    q := fromBits(n, p, rr, c)
                    if ¬balanced(q) {
                        continue
                    }
                    meet++
                    if xs, _ := squareCover(q, true); ¬xs.exists() {
                        if unsolvable[n] ≡ 0 {
                            first = q
                        }
                        unsolvable[n]++
                    }
                }
            }
        }
        desc := ""
        if unsolvable[n] > 0 {
            ⟨ Describe the first unsolvable instance 61 ⟩
        }
        fmt.Printf("␣␣␣␣␣␣␣␣n␣=␣%d:␣%d␣instances␣meet␣the␣condition,␣%d␣have␣no␣" +
            "solution%s\n", n, meet, unsolvable[n], desc)
    }
    claim(unsolvable[1] ≡ 0 ∧ unsolvable[2] > 0,
        "the␣smallest␣examples␣have␣n␣=␣2")
```

This code is used in section 58.

60. ⟨ Skip unless *rr* and *c* have the same column sums 60 ⟩ ≡

```
same := true
for k := 0; k < n; k++ {
    sr, sc := 0, 0
    for i := 0; i < n; i++ {
        sr += int(rr >> (i * n + k) & 1)
        sc += int(c >> (i * n + k) & 1)
    }
    same = same ∧ sr ≡ sc
}
if ¬same {
    continue
}
```

This code is used in section 59.

61. Instances are printed with their rows separated by slashes.

⟨ Describe the *first* unsolvable instance 61 ⟩ ≡

```
p := make([][]int, n)
for _, cell := range first.p {
    p[cell[0]] = append(p[cell[0]], cell[1])
}
desc = fmt.Sprintf(";\n␣␣␣␣␣␣␣␣␣␣the␣first␣is␣r␣=␣%s,␣c␣=␣%s,␣p␣=␣%s",
    matrix(n, first.r), matrix(n, first.c), matrix(n, p))
```

This code is used in section 59.

62. ⟨ Functions 6 ⟩ +≡

```
func matrix(n int, rows [][]int) string {
    var out []string
    for _, row := range rows {
        b := []byte(strings.Repeat("0", n))
        for _, k := range row {
            b[k] = '1'
        }
        out = append(out, string(b))
    }
    return strings.Join(out, "/")
}
```

63. The extension: answer 212(c). "Make $L(I,J) = \{1,\ldots,N\}$ for $M < I \le N$, $1 \le J \le N$. It is well known (Theorem 7.5.1L) that a latin rectangle can always be extended to a latin square." The answer takes the colors to be $1,\ldots,N$, as they are in answer 211. Two checks: random lists on small grids, colored by exact cover before and after, and the theorem itself, carried out by bipartite matching on random latin rectangles.

⟨ Functions 6 ⟩ +≡

```
func extendLists(L lists, N int) lists {
    E := slices.Clone(L)
    for I := len(L); I < N; I++ {
        row := make([][]int, N)
        for J := range row {
            for K := 0; K < N; K++ {
                row[J] = append(row[J], K)
            }
        }
        E = append(E, row)
    }
    return E
}
```

64. Each new row is a perfect matching between columns and the colors not yet in them. After M rows every column misses $N-M$ colors and every color is missing from $N-M$ columns, so the bipartite graph is regular and Hall's condition holds; augmenting paths find the matching.

⟨ Functions 6 ⟩ +≡

```
func extendRectangle(col [][]int, N int) [][]int {
    sq := slices.Clone(col)
    for len(sq) < N {
        used := make([][]bool, N)
        for J := range used {
            used[J] = make([]bool, N)
            for _, row := range sq {
                used[J][row[J]] = true
            }
        }
        owner := slices.Repeat([]int{-1}, N)
        var augment func(J int, seen []bool) bool
        augment = func(J int, seen []bool) bool {
            ⟨ Try to give column J a color, displacing others if necessary 65 ⟩
        }
        for J := 0; J < N; J++ {
            augment(J, make([]bool, N))
        }
        row := make([]int, N)
        for K, J := range owner {
            row[J] = K
        }
        sq = append(sq, row)
    }
    return sq
}
```

65. ⟨ Try to give column J a color, displacing others if necessary 65 ⟩ ≡

```
for K := 0; K < N; K++ {
    if used[J][K] ∨ seen[K] {
        continue
    }
    seen[K] = true
    if owner[K] < 0 ∨ augment(owner[K], seen) {
        owner[K] = J
        return true
    }
}
return false
```

This code is used in section 64.

66. Random lists: each color is on each list with probability *prob*.

⟨ Functions 6 ⟩ +≡

```
func randomLists(r *rand.Rand, M, N int, prob float64) lists {
    L := make(lists, M)
    for I := range L {
        L[I] = make([][]int, N)
        for J := range L[I] {
            for K := 0; K < N; K++ {
                if r.Float64() < prob {
                    L[I][J] = append(L[I][J], K)
                }
            }
        }
    }
    return L
}
```

67. ⟨ Check answer 212(c) 67 ⟩ ≡

```
fmt.Println("212(c). K_M x K_N to K_N x K_N")
r := rng(2123)
var nc, yes, badC, badExt int
for trial := 0; trial < 10 * reps; trial++ {
    N := 2 + r.IntN(4)
    M := 1 + r.IntN(N - 1)
    L := randomLists(r, M, N, 0.6)
    x, _ := colorCover(L, N)
    xe, _ := colorCover(extendLists(L, N), N)
    nc++
    if x.exists() {
        yes++
    }
    if x.exists() ≠ xe.exists() {
        badC++
    }
    ⟨ Extend a random latin rectangle to a square 68 ⟩
}
claim(badC ≡ 0, "%d grids up to 5 x 5, %d colorable: the extension agrees " +
    "every time, with %d exceptions", nc, yes, badC)
claim(badExt ≡ 0, "%d latin rectangles up to 29 x 30 extended by matching " +
    "to latin squares: %d failures", nc, badExt)
```

This code is used in section 3.

68. A random latin rectangle: a cyclic square with its rows, columns, and symbols permuted, cut to M rows.

⟨ Extend a random latin rectangle to a square 68 ⟩ ≡

```
Nr := 2 + r.IntN(29)
Mr := 1 + r.IntN(Nr - 1)
pr, pc, ps := r.Perm(Nr), r.Perm(Nr), r.Perm(Nr)
rect := make([][]int, Mr)
for I := range rect {
    rect[I] = make([]int, Nr)
    for J := range rect[I] {
        rect[I][J] = ps[(pr[I] + pc[J]) % Nr]
    }
}
if ¬validColoring(extendLists(Λ, Nr), Nr, extendRectangle(rect, Nr)) {
    badExt++
}
```

This code is used in section 67.

69. The square: answer 212(d). "Index everything by the set $\{1,\dots,N\} \cup \bigcup_{I,J}\{(I,J,K) \mid K \in L(I,J)\}$. The elements (I,J,K) where $K = \min L(I,J)$ are called *headers*. Set $p_{ij} = 1$ if and only if (i) $i = j = (I,J,K)$ is not a header; or (ii) $i = (I,J,K)$ is a header, and $j = J$ or $j = (I,J,K')$ is not a header; or (iii) $j = (I,J,K)$ is a header, and $i = I$ or $i = (I,J,K')$ is not a header. Set $r_{ik} = c_{ik} = 1$ if and only if (i) $1 \le i,k \le N$; or (ii) $i = (I,J,K)$ and $k = (I,J,K')$, and if i is not a header then ($K' = K$ or K' is the largest element $< K$ in $L(I,J)$)."

Take a header $h = (I,J,K)$. By (ii) the cell (h,J) must be filled. Its symbol k needs $r_{hk} = 1$, which by (ii) makes k an element (I,J,K'), and $c_{Jk} = 1$, which by (i) makes k an integer. No symbol is both, so as printed no instance with a nonempty list has a solution. The cell (I,h) has the same trouble.

If $k = (I,J,K')$ in the definition of r and c is read as $k = K'$, everything works. Row h then holds the symbols $L(I,J)$ in its cells (h,J) and (h,e) for the nonheaders e of the list; column h holds them in (I,h) and (e,h); and each nonheader $e = (I,J,K)$ has the two cells (e,e), (e,h) in its row and (e,e), (h,e) in its column for its two symbols K and its predecessor. So $X_{h,e} = X_{e,h}$ for every nonheader, the symbol missing from row h equals the one missing from column h, and $X_{h,J} = X_{I,h}$: that is the color of (I,J). Row I gets it once per J, column J once per I, so the colors form a list coloring; and conversely a color K_q in position q of the sorted list fills the gadget in exactly one way, with X_{h,e_s} equal to the predecessor K_{s-1} for $s \le q$ and to K_s for $s > q$.

square212 builds both versions; *literal* selects the printed one. Colors are $0,\dots,N-1$, and the element for position s of list $L(I,J)$ is $elem[I][J][s]$.

⟨ Functions 6 ⟩ +≡

```
func square212(L lists, N int, literal bool)(square, [][][]int) {
    n := N
    elem := make([][][]int, N)
    for I := range elem {
        elem[I] = make([][]int, N)
        for J := range elem[I] {
            for range L[I][J] {
                elem[I][J] = append(elem[I][J], n)
                n++
            }
        }
    }
    q := square{n: n, r: make([][]int, n), c: make([][]int, n)}
    for a := 0; a < N; a++ {
        for b := 0; b < N; b++ {
            q.r[a] = append(q.r[a], b)
        }
    }
    for I := 0; I < N; I++ {
        for J := 0; J < N; J++ {
            ⟨ Build the gadget for cell (I, J) 70 ⟩
        }
    }
    for i := range q.r {
        slices.Sort(q.r[i])
        q.c[i] = q.r[i]
    }
    return q, elem
}
```

70. ⟨ Build the gadget for cell (I, J) 70 ⟩ ≡

```
l, e := L[I][J], elem[I][J]
if len(l) ≡ 0 {
    continue
}
h := e[0]
q.p = append(q.p, [2]int{h, J}, [2]int{I, h})
for s := 1; s < len(l); s++ {
    q.p = append(q.p, [2]int{e[s], e[s]}, [2]int{h, e[s]}, [2]int{e[s], h})
}
for s := range l {
    for t := range l {
        if s ≡ 0 ∨ t ≡ s ∨ t ≡ s − 1 {
            if literal {
                q.r[e[s]] = append(q.r[e[s]], e[t])
            } else {
                q.r[e[s]] = append(q.r[e[s]], l[t])
            }
        }
    }
}
```

This code is used in section 69.

71. The forward direction by hand, as described above, and a check of any filling against the tensor reading.

⟨ Functions 6 ⟩ +≡

```
func fillSquare(L lists, N int, elem [][][]int, col [][]int) map[[2]int]int {
    X := map[[2]int]int{}
    for I := 0; I < N; I++ {
        for J := 0; J < N; J++ {
            l, e := L[I][J], elem[I][J]
            q := slices.Index(l, col[I][J])
            X[[2]int{e[0], J}], X[[2]int{I, e[0]}] = l[q], l[q]
            for s := 1; s < len(l); s++ {
                k, other := l[s], l[s − 1]
                if s ≤ q {
                    k, other = other, k
                }
                X[[2]int{e[0], e[s]}], X[[2]int{e[s], e[0]}] = k, k
                X[[2]int{e[s], e[s]}] = other
            }
        }
    }
    return X
}
```

72. ⟨ Functions 6 ⟩ +≡

```
func validSquare(q square, X map[[2]int]int) bool {
    if len(X) ≠ len(q.p) {
        return false
    }
    rows, cols := map[[2]int]int{}, map[[2]int]int{}
    for _, cell := range q.p {
        k, ok := X[cell]
        _, inR := slices.BinarySearch(q.r[cell[0]], k)
        _, inC := slices.BinarySearch(q.c[cell[1]], k)
        if ¬ok ∨ ¬inR ∨ ¬inC {
            return false
        }
        rows[[2]int{cell[0], k}]++
        cols[[2]int{cell[1], k}]++
    }
    for i := 0; i < q.n; i++ {
        if ¬exactlyOnce(rows, i, q.r[i]) ∨ ¬exactlyOnce(cols, i, q.c[i]) {
            return false
        }
    }
    return true
}
func exactlyOnce(used map[[2]int]int, i int, want []int) bool {
    for _, k := range want {
        if used[[2]int{i, k}] ≠ 1 {
            return false
        }
    }
    return true
}
```

73. The checks run on random lists for grids up to 3×3. Each color is on a list with probability 0.7, so now and then a list is empty, and the array reading of part (a) is watched for that case.

⟨ Check answer 212(d) 73 ⟩ ≡

```
fmt.Println("212(d). K_N x K_N to a partial latin square")
r := rng(2124)
var nd, colorable, withEmpty, badSize, litSolved, litSymbol, badCount,
arrayNonempty, arrayEmpty, badFill, badDecode int
for trial := 0; trial < 10 * reps; trial++ {
    N := 1 + r.IntN(3)
    L := randomLists(r, N, N, 0.7)
    nonempty := true
    sum := 0
    for I := range L {
        for J := range L[I] {
            nonempty = nonempty ∧ len(L[I][J]) > 0
            sum += len(L[I][J])
        }
    }
    nd++
    ⟨ Try the square as printed 74 ⟩
    ⟨ Try the square with k = K′ 75 ⟩
}
claim(badSize ≡ 0, "%d grids: n = N + sum |L(I,J)| every time", nd)
claim(litSolved ≡ 0 ∧ litSymbol ≡ 0, "as printed, no square has a solution, " +
    "and in every grid with nonempty lists some cell (h,J) has no symbol")
claim(badCount ≡ 0, "with k = K', the solutions of the tensor are exactly as " +
    "many as the colorings (%d grids colorable): %d exceptions", colorable,
    badCount)
claim(badFill ≡ 0 ∧ badDecode ≡ 0, "every coloring fills the square by " +
    "hand, and every solution decodes to a coloring: %d and %d exceptions",
    badFill, badDecode)
claim(arrayNonempty ≡ 0, "under the array reading the counts also agree " +
    "when no list is empty; they differ in %d of the %d grids with an empty " +
    "list", arrayEmpty, withEmpty)
```

This code is used in section 3.

74. ⟨ Try the square as printed 74 ⟩ ≡

```
lit, lelem := square212(L, N, true)
if lit.n ≠ N + sum {
    badSize++
}
xl, _ := squareCover(lit, true)
if xl.exists() {
    litSolved++
}
if nonempty {
    h := lelem[0][0][0]
    if slices.ContainsFunc(lit.r[h], func(k int) bool {
        _, ok := slices.BinarySearch(lit.c[0], k)
        return ok
    }) {
        litSymbol++
    }
}
```

This code is used in section 73.

75. ⟨ Try the square with $k = K'$ 75 ⟩ ≡

```
fix, elem := square212(L, N, false)
xc, _ := colorCover(L, N)
colors := xc.total()
if colors > 0 {
    colorable++
}
xt, triples := squareCover(fix, true)
if xt.total() ≠ colors {
    badCount++
}
xa, _ := squareCover(fix, false)
if xa.total() ≠ colors {
    if nonempty {
        arrayNonempty++
    } else {
        arrayEmpty++
    }
}
if ¬nonempty {
    withEmpty++
}
colorings(L, N, func(col [][]int) bool {
    if ¬validSquare(fix, fillSquare(L, N, elem, col)) {
        badFill++
    }
    return true
})
xt.search(func(sol []int) bool {
    ⟨ Decode a solution of the square into a coloring 76 ⟩
    return true
})
```

This code is used in section 73.

76. ⟨ Decode a solution of the square into a coloring 76 ⟩ ≡

```
X := map[[2]int]int{}
for _, o := range sol {
    X[[2]int{triples[o][0], triples[o][1]}] = triples[o][2]
}
col := make([][]int, N)
for I := range col {
    col[I] = make([]int, N)
    for J := range col[I] {
        h := elem[I][J][0]
        col[I][J] = X[[2]int{h, J}]
        if X[[2]int{I, h}] ≠ col[I][J] {
            badDecode++
        }
    }
}
if ¬validColoring(L, N, col) {
    badDecode++
}
```

This code is used in section 75.

77. The whole chain. Now the reductions are composed: a problem in which every variable occurs four times, on three variables, becomes the lists of answer 211 for $K_{24} \square K_3$, the lists of answer 212(c) for $K_{24} \square K_{24}$, and the square of answer 212(d), read with $k = K'$, which has $n = 24 + \sum |L(I, J)|$, a little over twelve thousand. The package solves the square; the solution is decoded to a coloring of $K_{24} \square K_{24}$, whose first three rows decode to an assignment, which must satisfy the clauses. And in the other direction, without the package, a solution found by brute force is turned into a coloring of the grid by hand, extended to a latin square by matching, and filled into the partial latin square, which is checked against the definition.

Unsatisfiable problems of this kind have at least sixteen clauses, which would make n about 857,000. So the chain is also run on random lists for $K_4 \square K_3$ through $K_6 \square K_3$, many of them uncolorable, and the package's verdict on the square is compared with exact cover on the grid.

```
⟨ Run the whole chain through the sat package 77 ⟩ ≡
    fmt.Println("211␣and␣212.␣from␣3SAT␣to␣a␣partial␣latin␣square")
    r := rng(212)
    var nw, badW, badBack, badForward, sizeN, sizeOpts, sizeClauses int
    for trial := 0; trial < 3; trial++ {
        f := fourTimes(r, 3, true)
        sigma := cycles(f, r, false)
        L := grid211(f, sigma)
        N := 6 * len(f.cl)
        E := extendLists(L, N)
        q, elem := square212(E, N, false)
        nw++
        ⟨ Solve the square with the package 78 ⟩
        if ¬ok {
            badW++
            continue
        }
        ⟨ Decode the model back to the clauses 81 ⟩
        ⟨ Go forward by hand from a solution of the clauses 82 ⟩
    }
    claim(badW ≡ 0 ∧ badBack ≡ 0, "%d␣problems:␣the␣square␣(n␣=␣%d,␣%d␣" +
        "options,␣%d␣clauses)␣is␣solved,␣and␣decodes␣to␣a␣solution␣of␣the␣clauses",
        nw, sizeN, sizeOpts, sizeClauses)
    claim(badForward ≡ 0, "a␣solution␣of␣the␣clauses␣fills␣the␣square␣by␣hand:␣" +
        "%d␣exceptions", badForward)
    ⟨ Run small grids through the whole chain 83 ⟩
```

This code is used in section 3.

78. One variable per triple (i, j, k) with $p_{ij} = r_{ik} = c_{jk} = 1$, and exactly one true in each cell, each row–symbol pair, and each column–symbol pair.

⟨ Solve the square with the package 78 ⟩ ≡

```
s := sat.New()
groups := map[[3]int][]sat.Lit{}
var triples [][3]int
var tlits []sat.Lit
for _, cell := range q.p {
    i, j := cell[0], cell[1]
    for _, k := range q.r[i] {
        if _, in := slices.BinarySearch(q.c[j], k); in {
            l := s.NewVar()
            triples, tlits = append(triples, [3]int{i, j, k}), append(tlits, l)
            for _, g := range [][3]int {{0, i, j}, {1, i, k}, {2, j, k} }{
                groups[g] = append(groups[g], l)
            }
        }
    }
}
⟨ Require exactly one true literal in each group 79 ⟩
st, _ := s.Solve(context.Background())
ok := st ≡ sat.Sat ∧ complete
sizeN, sizeOpts, sizeClauses = q.n, len(triples), s.NumClauses()
```

This code is used in sections 77, 83.

79. A group that no triple reaches makes the square unsolvable at once.

⟨ Require exactly one true literal in each group 79 ⟩ ≡

```
complete := true
for _, cell := range q.p {
    complete = complete ∧ groups[[3]int{0, cell[0], cell[1]}] ≠ Λ
}
for i := 0; i < q.n; i++ {
    for _, k := range q.r[i] {
        complete = complete ∧ groups[[3]int{1, i, k}] ≠ Λ
    }
    for _, k := range q.c[i] {
        complete = complete ∧ groups[[3]int{2, i, k}] ≠ Λ
    }
}
for _, ls := range groups {
    s.AddClause(ls...)
    atMostOne(s, ls)
}
```

This code is used in section 78.

80. The color of (I, J) is the symbol in the cell (h, J) of its header.

```
⟨ Read X and col off the model 80 ⟩ ≡
    X := map[[2]int]int{ }
    for t, l := range tlits {
        if s.Value(l) {
            X[[2]int{triples[t][0], triples[t][1]}] = triples[t][2]
        }
    }
    col := make([][]int, N)
    for I := range col {
        col[I] = make([]int, N)
        for J := range col[I] {
            col[I][J] = X[[2]int{elem[I][J][0], J}]
        }
    }
```

This code is used in sections 81, 84.

81.

```
⟨ Decode the model back to the clauses 81 ⟩ ≡
    ⟨ Read X and col off the model 80 ⟩
    if ¬validSquare(q, X) ∨ ¬validColoring(E, N, col) ∨
        ¬satisfies(f, assignment(f, col[:3])) {
        badBack++
    }
```

This code is used in section 77.

82.

```
⟨ Go forward by hand from a solution of the clauses 82 ⟩ ≡
    a := models(f)[0]
    latin := extendRectangle(colorFromModel(f, sigma, a), N)
    if ¬validSquare(q, fillSquare(E, N, elem, latin)) {
        badForward++
    }
```

This code is used in section 77.

83. For the small grids the package's verdict is compared with exact cover on the $K_M \square K_3$ lists, and a solution is decoded once more.

⟨ Run small grids through the whole chain 83 ⟩ ≡

```
var nSm, yesSm, badSm int
for trial := 0; trial < reps/4; trial++ {
    N := 4 + trial % 3
    L := randomLists(r, 3, N, 0.45)
    x, _ := colorCover(L, N)
    want := x.exists()
    if want {
        yesSm++
    }
    E := extendLists(L, N)
    q, elem := square212(E, N, false)
    nSm++
    ⟨ Solve the square with the package 78 ⟩
    if ok ≠ want {
        badSm++
    } else if ok {
        ⟨ Decode the square of a small grid 84 ⟩
    }
}
claim(badSm ≡ 0, "%d␣random␣grids␣K_4␣x␣K_3␣to␣K_6␣x␣K_3,␣%d␣colorable:␣" +
    "the␣package␣agrees␣on␣every␣square,␣and␣its␣solutions␣decode;␣" +
    "%d␣exceptions", nSm, yesSm, badSm)
```

This code is used in section 77.

84. ⟨ Decode the square of a small grid 84 ⟩ ≡

```
⟨ Read X and col off the model 80 ⟩
if ¬validSquare(q, X) ∨ ¬validColoring(E, N, col) {
    badSm++
}
```

This code is used in section 83.

85. Index.

a: 9, 19, 25, 26, 38, 39, 40, 43, 44, 49, 55, 69, 82.
add: 13, 34, 53.
AddClause: 11, 35, 79.
answer207: 20, 22, 25, 44.
append: 9, 13, 18, 20, 21, 22, 23, 27, 32, 33, 34, 35, 40, 41, 44, 47, 53, 54, 61, 62, 63, 64, 66, 69, 70, 78.
args: 6.
arrayEmpty: 73, 75.
arrayNonempty: 73, 75.
as: 11.
assignment: 38, 43, 44, 81.
assumptions: 11.
atMostOne: 35, 79.
Atoi: 9.
augment: 64, 65.
a207: 25, 26.
b: 14, 16, 19, 62, 69.
Background: 11, 15, 35, 78.
bad: 49.
badBack: 77, 81.
badBig: 44.
badC: 67.
badCount: 43, 73, 75.
badDecode: 73, 76.
badExt: 67, 68.
badFill: 73, 75.
badFirst: 45.
badForward: 77, 82.
badHand: 43.
badLeft: 45.
badNec: 56, 57.
badOut: 50.
badPad: 27, 29.
badPair: 47, 48.
badRight: 45.
badSat: 27.
badShape: 27, 28.
badSize: 73, 74.
badSm: 83, 84.
badSym: 56, 57.
badW: 77.
balanced: 55, 56, 57, 58, 59.
before: 22, 27, 28, 29.
BinarySearch: 53, 58, 72, 74, 78.
bits: 56.
Builder: 16.
c: 9, 10, 11, 19, 22, 40, 41, 43, 47, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 69, 72, 74, 78, 79.
cancel: 15.
cell: 52, 53, 55, 61, 72, 78, 79.
cells: 3, 15.
ck: 55.
cl: 8, 9, 10, 11, 18, 20, 21, 22, 23, 25, 26, 27, 28, 29, 31, 32, 33, 38, 39, 40, 41, 43, 44, 45, 47, 48, 50, 77.
claim: 6, 24, 25, 26, 27, 43, 44, 45, 47, 49, 50, 56, 58, 59, 67, 73, 77, 83.
Clone: 22, 26, 43, 63, 64.
clones208: 22, 27, 44, 47.
cnf: 8, 9, 10, 11, 18, 20, 21, 22, 25, 26, 27, 28, 31, 33, 38, 39, 41, 44, 47.
col: 34, 36, 37, 38, 39, 40, 43, 44, 45, 64, 71, 75, 76, 80, 81, 84.
colorable: 73, 75.
colorCover: 34, 43, 49, 50, 67, 75, 83.
colorFromModel: 39, 43, 82.
colorings: 34, 45, 75.
colors: 75.
colorSAT: 35, 44, 48, 49.
cols: 35, 72.
complete: 78, 79.
Contains: 37, 58.
ContainsFunc: 9, 26, 74.
context: 11, 15, 35, 78.
copy: 40.
Count: 9, 17.
cs: 9.
ct: 57.
ctx: 15.
cube: 44.
cube208: 44.
cycles: 33, 43, 44, 45, 48, 49, 50, 77.
c31: 58.
c32: 58.
d: 22, 26.
Dance: 15, 17.
Delete: 26.
DeleteFunc: 43.
desc: 59, 61.
differ: 56, 57.
distinct: 41.
dlx: 14, 15, 17.
drop: 22.

E: 63, 77, 81, 82, 83, 84.
e: 70, 71.
elem: 69, 70, 71, 75, 76, 77, 80, 82, 83.
entries: 58.
ex: 58.
exact: 52.
exactlyOnce: 72.
exists: 17, 49, 50, 57, 58, 59, 67, 74, 83.
Exit: 7.
extendLists: 63, 67, 68, 77, 83.
extendRectangle: 64, 68, 82.
extra: 22.
f: 9, 10, 11, 18, 20, 21, 22, 27, 31, 32, 33, 38, 39, 40, 41, 43, 44, 45, 47, 48, 50, 77, 81, 82.
failures: 5, 6, 7.
Fields: 9.
fillSquare: 71, 75, 82.
first: 40, 59, 61.
fix: 75.
flag: 4.
Float64: 66.
fmt: 6, 7, 16, 24, 25, 27, 42, 56, 58, 59, 61, 67, 73, 77.
forced: 49.
format: 6.
found: 17.
four: 49.
fours: 25.
fourTimes: 41, 43, 44, 45, 50, 77.
Fprintf: 16.
free: 40.
fromBits: 54, 56, 57, 58, 59.
g: 18, 22, 23, 25, 27, 28, 29, 44, 47, 48, 78.
gadget: 19, 20, 21, 25.
good: 28.
grid211: 31, 43, 44, 45, 48, 49, 50, 77.
group: 35.
groups: 78, 79.
h: 18, 70, 74, 76.
holds: 9, 26, 40, 43.
I: 31, 34, 35, 36, 37, 63, 66, 68, 69, 70, 71, 73, 76, 80.
i: 11, 15, 16, 18, 22, 23, 29, 33, 35, 41, 52, 53, 54, 55, 57, 60, 69, 72, 78, 79.
id: 52, 53.
in: 78.
inC: 72.
Index: 71.
index: 14, 15, 16.
inR: 72.
inside: 33.
inSigma: 49.
IntN: 27, 29, 41, 47, 67, 68, 73.
IntVar: 4.
Int64: 17.
it: 16, 25.
items: 12, 13, 16.
Itoa: 9, 16.
iwamaTakaki: 21, 25, 44.
i2: 18.
J: 31, 34, 35, 36, 37, 63, 64, 65, 66, 68, 69, 70, 71, 73, 76, 80.
j: 23, 33, 35, 41, 53, 54, 57, 78.
Join: 9, 15, 16, 62.
K: 34, 35, 37, 63, 64, 65, 66.
k: 18, 27, 32, 40, 47, 52, 53, 55, 60, 62, 71, 72, 74, 78, 79.
L: 31, 32, 34, 35, 36, 37, 43, 44, 63, 66, 67, 69, 70, 71, 73, 74, 75, 76, 77, 83.
l: 9, 10, 11, 26, 29, 32, 35, 36, 43, 70, 71, 78, 80.
latin: 82.
left: 45.
lelem: 74.
len: 11, 15, 16, 18, 22, 25, 26, 27, 28, 29, 31, 33, 34, 35, 38, 39, 40, 41, 43, 44, 45, 48, 49, 50, 52, 53, 55, 63, 64, 70, 71, 72, 73, 77, 78.
line: 16.
lists: 30, 31, 34, 35, 37, 63, 66, 69, 71.
lit: 35, 36, 74.
Lit: 11, 35, 78.
literal: 69, 70.
litSolved: 73, 74.
litSymbol: 73, 74.
ls: 9, 11, 35, 41, 79.
M: 34, 35, 36, 66, 67.
m: 6, 18, 22, 23, 27, 28, 29, 31, 32, 33, 39, 40, 45.
main: 3.
make: 10, 11, 15, 16, 18, 31, 33, 34, 35, 36, 37, 39, 54, 55, 59, 61, 63, 64, 66, 68, 69, 76, 80.
mark: 6.
matrix: 61, 62.
Max: 25.
meet: 59.
Min: 25.
mode: 4, 5, 6.
models: 9, 25, 26, 27, 43, 48, 49, 50, 82.
Mr: 68.

ms: 9, 26, 49.
N: 31, 34, 35, 36, 37, 43, 63, 64, 65, 66, 67, 69, 71, 73, 74, 75, 76, 77, 80, 81, 82, 83, 84.
n: 8, 9, 10, 11, 18, 20, 21, 22, 25, 26, 27, 28, 33, 41, 43, 44, 47, 51, 52, 54, 55, 56, 57, 59, 60, 61, 62, 69, 72, 74, 78, 79.
name: 44.
names: 16.
nb: 56, 59.
nBig: 44.
nc: 67.
nCol: 45.
nd: 73.
New: 6, 11, 17, 35, 78.
newCover: 13, 34, 53.
NewPCG: 6.
NewReader: 15, 17.
NewVar: 11, 35, 78.
NewXCC: 15.
nInside: 47, 48.
nonempty: 73, 74, 75.
Not: 35.
nOut: 50.
Nr: 68.
nSix: 43.
nSm: 83.
nSmall: 43.
NumClauses: 78.
nw: 77.
n208: 27, 47.
o: 15, 16, 33, 34, 76.
occ: 10, 18, 25, 33, 49.
occA: 28.
occB: 28.
occurrences: 10, 25, 28, 49.
off: 22.
ok: 6, 29, 41, 44, 48, 53, 72, 74, 77, 78, 83.
okDrop: 26.
okIn: 49.
okOut: 49.
on: 3, 6.
one: 56.
opts: 12, 13, 15, 16.
os: 7.
other: 71.
out: 62.
owner: 64, 65.
p: 51, 52, 53, 54, 55, 56, 57, 58, 59, 61, 70, 72, 78, 79.
Parse: 4.
parseCNF: 9, 24, 49.
pb: 58.
pc: 68.
Perm: 68.
pi: 55.
pj: 55.
pr: 68.
primary: 12, 13, 15, 16, 17, 52, 53.
Printf: 6, 7, 59.
Println: 7, 24, 25, 27, 42, 56, 58, 67, 73, 77.
prob: 66.
prod: 43.
proj: 25.
ps: 68.
q: 31, 32, 33, 38, 39, 40, 45, 48, 50, 52, 53, 54, 55, 56, 57, 59, 69, 70, 71, 72, 77, 78, 79, 81, 82, 83, 84.
r: 22, 27, 29, 33, 41, 43, 44, 45, 47, 48, 49, 50, 51, 52, 53, 54, 55, 58, 61, 66, 67, 68, 69, 70, 72, 73, 74, 77, 78, 79, 83.
Rand: 6, 33, 41, 66.
rand: 6, 33, 41, 66.
randomLists: 66, 67, 73, 83.
rc: 58.
rect: 68.
reduced: 22, 23.
Repeat: 9, 62, 64.
reps: 4, 5, 27, 43, 44, 45, 47, 50, 67, 73, 83.
res: 15.
rest: 26.
right: 45.
rk: 55.
rng: 6, 27, 43, 67, 73, 77.
row: 62, 63, 64.
rows: 35, 62, 72.
rr: 56, 57, 59, 60.
s: 9, 11, 31, 32, 35, 36, 39, 69, 70, 71, 78, 79, 80.
same: 29, 41, 60.
Sat: 11, 35, 78.
sat: 11, 35, 78.
satisfies: 9, 44, 81.
sc: 60.
search: 15, 17, 34, 75.
seed: 4, 5, 6.
seen: 37, 64, 65.

sepa: 50.
sg: 18.
sg2: 18.
Shuffle: 33, 41.
sigma: 30, 31, 32, 33, 39, 43, 45, 48, 50, 77, 82.
sizeClauses: 77, 78.
sizeN: 77, 78.
sizeOpts: 77, 78.
slices: 9, 22, 25, 26, 31, 37, 43, 53, 58, 63, 64, 69, 71, 72, 74, 78.
sol: 15, 34, 75, 76.
Solutions: 15.
Solve: 11, 35, 78.
solveCNF: 11, 27, 44.
Solver: 35.
Sort: 31, 69.
Split: 9.
Sprintf: 6, 61.
sq: 64.
square: 51, 52, 54, 55, 59, 69, 72.
squareCover: 52, 56, 57, 58, 59, 74, 75.
square212: 69, 74, 75, 77, 83.
sr: 60.
st: 11, 35, 78.
stays: 33.
strconv: 9, 16.
stream: 6.
String: 9, 14, 24.
strings: 9, 15, 16, 17, 62.
StringVar: 4.
sum: 73, 74.
t: 16, 22, 24, 32, 36, 40, 41, 70, 80.
tensor: 57.
text: 15, 17.
tlits: 78, 80.
total: 17, 43, 56, 57, 75.
tovey: 18, 22, 24.
trial: 27, 43, 44, 45, 47, 50, 67, 73, 77, 83.
TrimSuffix: 9.
triples: 34, 52, 53, 75, 76, 78, 80.
try: 33, 48.
u: 23.
Uint64Var: 4.
unsolvable: 59.
up: 23.
used: 64, 65, 72.
v: 9, 11, 18, 25, 28, 29, 41, 49.
val: 29.
validColoring: 37, 43, 44, 68, 76, 81, 84.
validSquare: 72, 75, 81, 82, 84.
Value: 36, 80.
value: 39, 40.
visit: 15, 34.
w: 9, 32, 39.
want: 43, 72, 83.
wantNeg: 28.
wantPos: 28.
WithCancel: 15.
WithContext: 15.
withEmpty: 73, 75.
withPair: 47.
WriteByte: 16.
WriteString: 16.
X: 71, 72, 76, 80, 81, 84.
x: 9, 13, 14, 15, 16, 17, 19, 34, 43, 49, 50, 52, 53, 67, 83.
xa: 56, 57, 75.
xb: 58.
xc: 75.
xcover: 12, 13, 14, 15, 17, 34, 52.
xe: 67.
xl: 74.
xs: 57, 59.
xt: 56, 57, 75.
x1: 56.
x2: 56.
y: 19.
yes: 67.
yesSm: 83.
z: 19.
zdd: 3, 17.

⟨ Build the gadget for cell (I, J) 70 ⟩ Used in section 69
⟨ Check answer 207 25 ⟩ Used in section 24
⟨ Check answer 208 27 ⟩ Used in section 24
⟨ Check answer 211 42 ⟩ Used in section 3
⟨ Check answer 212(a) 56 ⟩ Used in section 3
⟨ Check answer 212(b) 58 ⟩ Used in section 3
⟨ Check answer 212(c) 67 ⟩ Used in section 3
⟨ Check answer 212(d) 73 ⟩ Used in section 3
⟨ Check answers 204(a), 207, and 208 24 ⟩ Used in section 3
⟨ Check that answer 208 produces such clauses 47 ⟩ Used in section 46
⟨ Check that cycles which leave the clause always work 50 ⟩ Used in section 46
⟨ Check that every clause of answer 207 is necessary 26 ⟩ Used in section 25
⟨ Check that the padding can always be satisfied 29 ⟩ Used in section 27
⟨ Check the example in parentheses 45 ⟩ Used in section 42
⟨ Check the shape of g 28 ⟩ Used in section 27
⟨ Color the grid of g with several cycles 48 ⟩ Used in section 47
⟨ Compare the readings, the symmetry, and the necessary condition 57 ⟩ Used in section 56
⟨ Count the colorings of small grids 43 ⟩ Used in section 42
⟨ Decide larger grids with the package 44 ⟩ Used in section 42
⟨ Decode a solution of the square into a coloring 76 ⟩ Used in section 75
⟨ Decode the model back to the clauses 81 ⟩ Used in section 77
⟨ Decode the square of a small grid 84 ⟩ Used in section 83
⟨ Describe the *first* unsolvable instance 61 ⟩ Used in section 59
⟨ Extend a random latin rectangle to a square 68 ⟩ Used in section 67
⟨ Find the smallest examples 59 ⟩ Used in section 58
⟨ Functions 6 ⟩ Used in section 3
⟨ Give the vertex $3m + q$ its three lists 32 ⟩ Used in section 31
⟨ Global variables 5 ⟩ Used in section 3
⟨ Go forward by hand from a solution of the clauses 82 ⟩ Used in section 77
⟨ Hand out the free colors of column 1, clause by clause 40 ⟩ Used in section 39
⟨ Look at clauses that hold a variable twice 46 ⟩ Used in section 42
⟨ Make the options of the square 53 ⟩ Used in section 52
⟨ Pad with the variables u_i 23 ⟩ Used in section 22
⟨ Read the coloring off the model 36 ⟩ Used in section 35
⟨ Read the command line 4 ⟩ Used in section 3
⟨ Read X and *col* off the model 80 ⟩ Used in sections 81, 84
⟨ Require exactly one true literal in each group 79 ⟩ Used in section 78
⟨ Run small grids through the whole chain 83 ⟩ Used in section 77
⟨ Run the whole chain through the `sat` package 77 ⟩ Used in section 3
⟨ Say how it went 7 ⟩ Used in section 3
⟨ Show a problem where a step inside a clause breaks the reduction 49 ⟩ Used in section 46
⟨ Skip unless *rr* and *c* have the same column sums 60 ⟩ Used in section 59
⟨ Solve the square with the package 78 ⟩ Used in sections 77, 83
⟨ Try the square as printed 74 ⟩ Used in section 73
⟨ Try the square with $k = K'$ 75 ⟩ Used in section 73

⟨ Try to give column J a color, displacing others if necessary 65 ⟩ Used in section 64
⟨ Types 8 ⟩ Used in section 3
⟨ Write x in DLX format, and index its options by their text 16 ⟩ Used in section 14

Partial latin square construction

	Section	Page
Introduction	1	1
Clauses	8	4
Exact cover	12	7
The ladder: answers 204(a), 207, and 208	18	10
The grid: answer 211	30	16
Clauses with a variable twice	46	27
The tensor: answer 212(a)	51	30
The small example: answer 212(b)	58	34
The extension: answer 212(c)	63	37
The square: answer 212(d)	69	40
The whole chain	77	47
Index	85	51